



Computer Vision for Autonomous Vehicles

*Week 7-8-9: Feature Detection and 2-view
geometry*

Instructor: Golnaz Habibi, PhD

University of Oklahoma

Fall 2024



Announcement

- Lab 3 due I next Monday September
- **We had first tutorial session today 11:30- 1:15**
- Instruction for running the cars will be posted today

- Today:
- Feature descriptor
- feature matching
- 2View Geometry

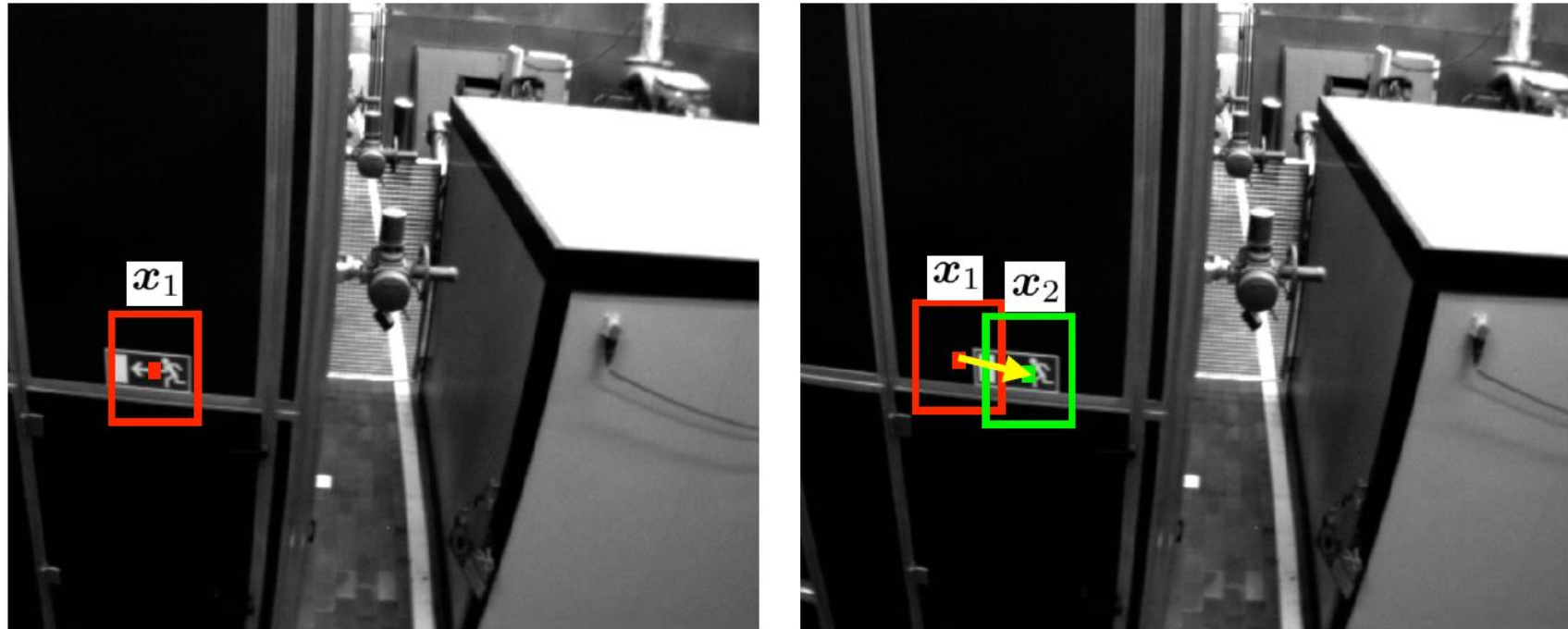
Correspondence

given a corner in image I_1 (and its neighborhood),
how can we find corresponding pixel in I_2 ?



- **Feature tracking** (~ optical flow)
- **Feature matching** (descriptor-based)

Feature Tracking

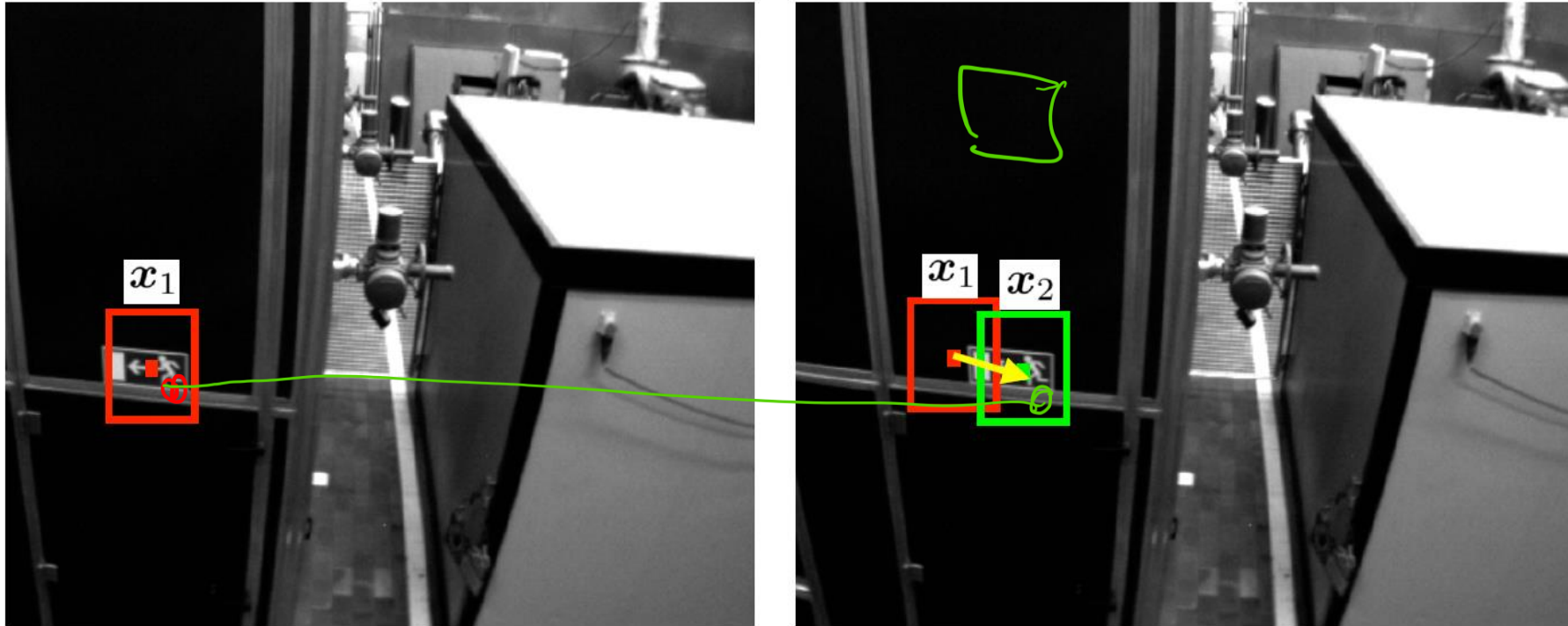


Computing the corresponding pixel (x_2) is the same as computing the displacement δ

$$x_2 = x_1 + \delta$$

(translational motion model)

Feature Tracking



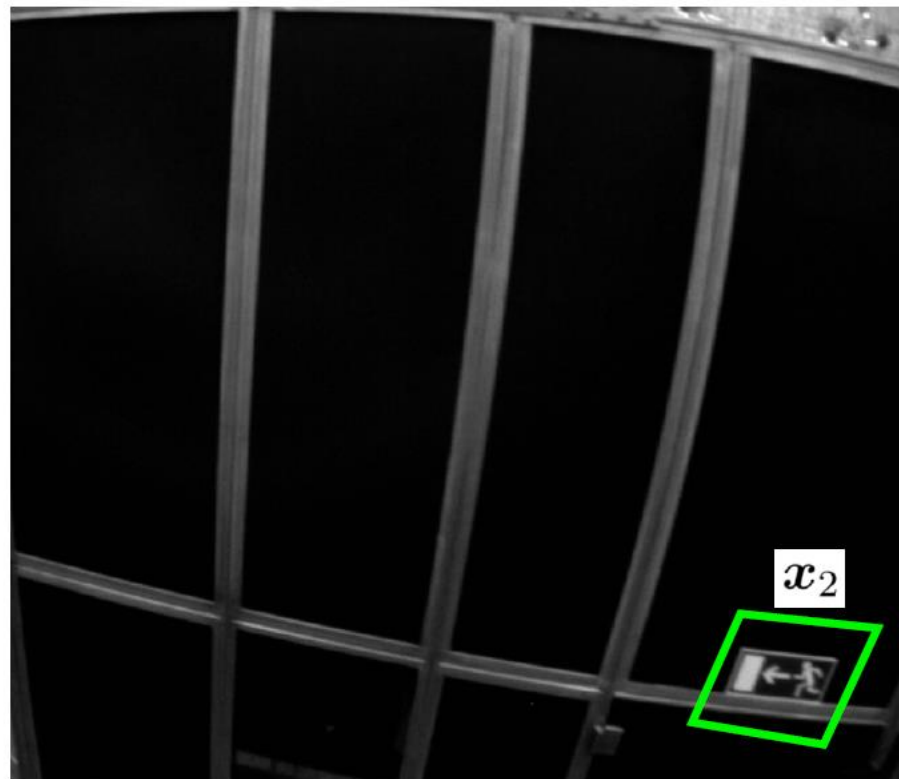
Computing the corresponding pixel (x_2) is the same as computing the displacement δ

Idea: find the displacement δ such that the intensity of points in the neighborhood of y in I_1 is similar to the one in I_2

$$\min_{\delta} \sum_{y \in W(x_1)} \|\mathcal{I}_1(y) - \mathcal{I}_2(y + \delta)\|^2$$

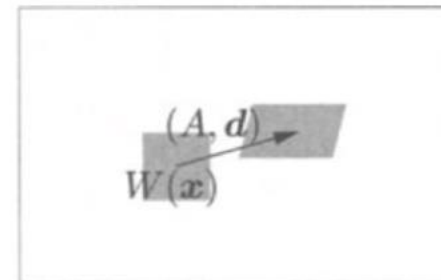
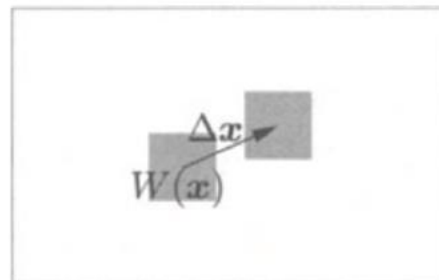
(translational motion model)

Feature Tracking

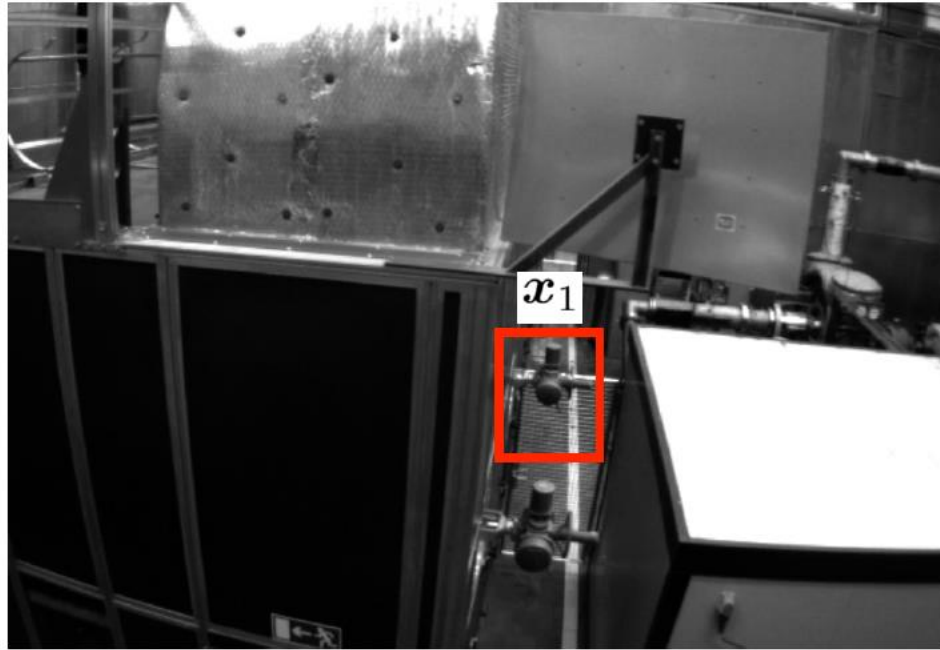


$$\min_{A, \delta} \sum_{y \in W(x_1)} \|\mathcal{I}_1(y) - \mathcal{I}_2(Ay + \delta)\|^2$$

(affine motion model)

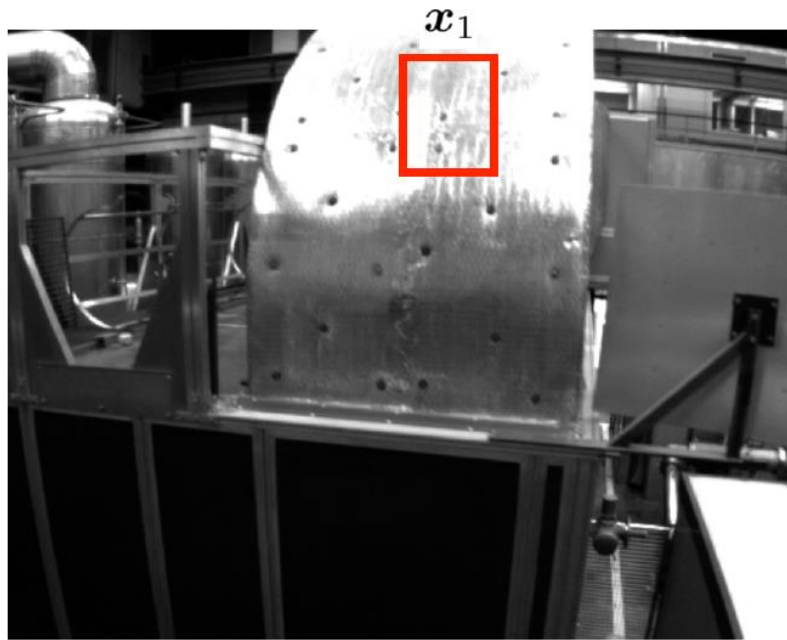


Hidden Assumptions...



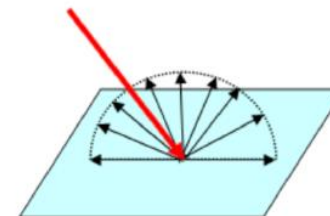
Pixel motion models not valid in presence of occlusions

Hidden Assumptions...



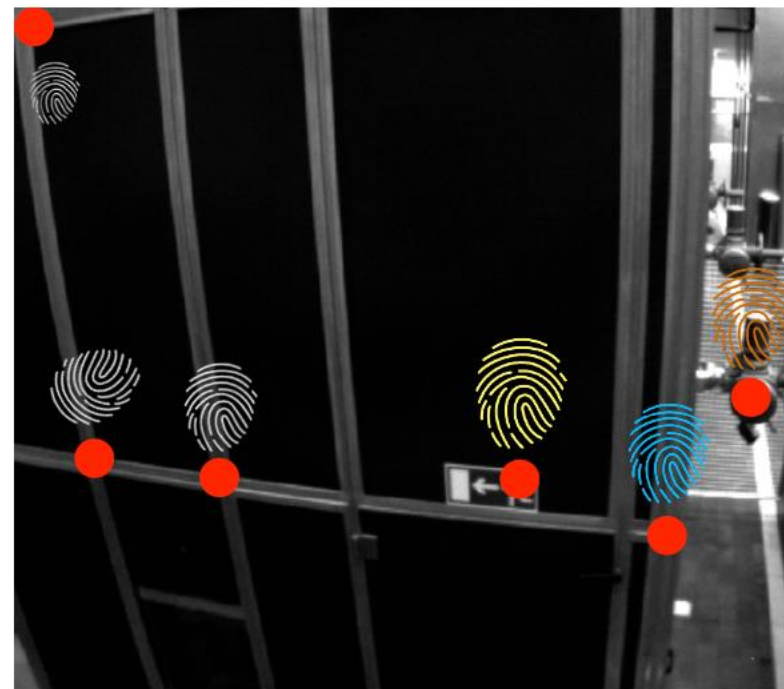
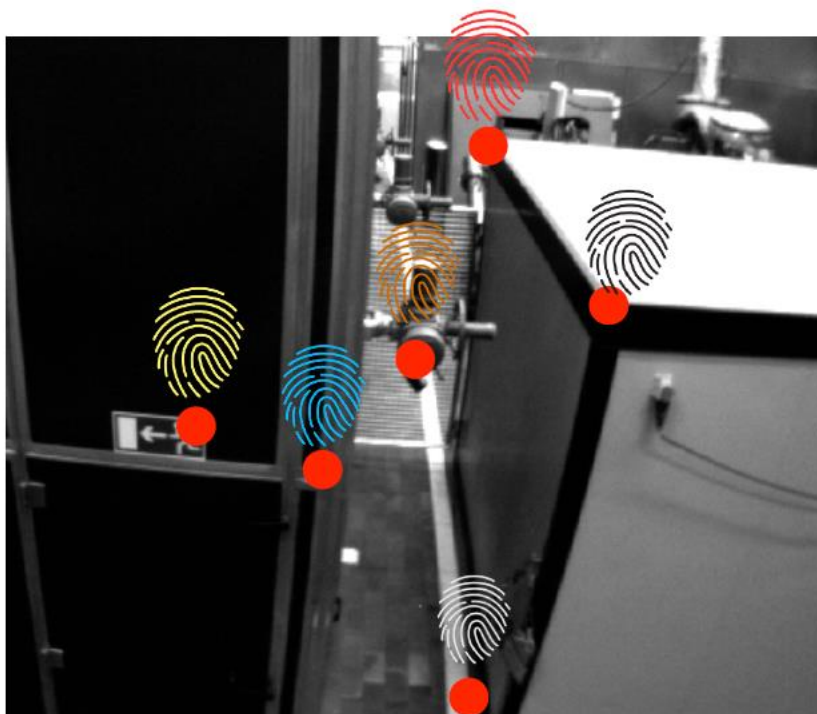
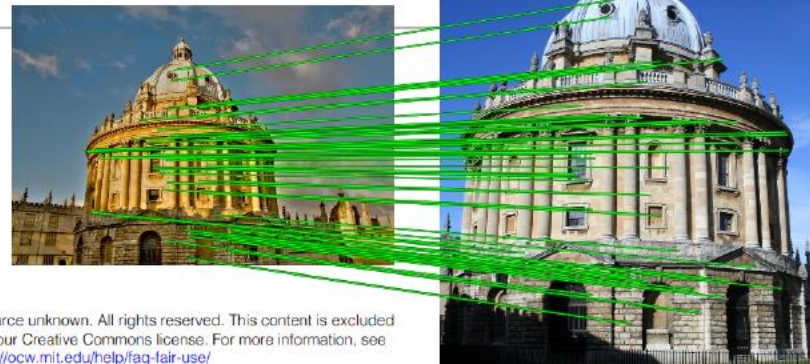
Matching image patches assume that the brightness
does not change due to viewpoint changes
(**brightness constancy constraints**)

True for Lambertian surfaces



Descriptor-based Feature Matching

Feature tracking does not typically work for large changes of viewpoint
(**large baseline**)

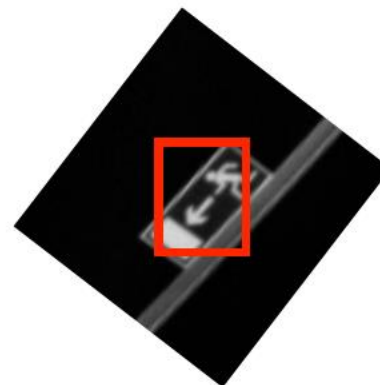
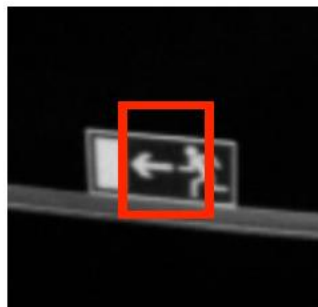


Descriptor is a signature we attach to a (point) feature,

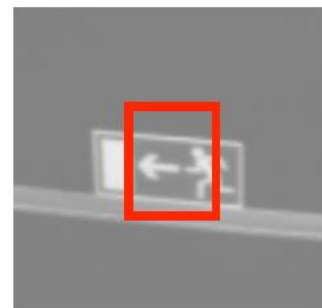
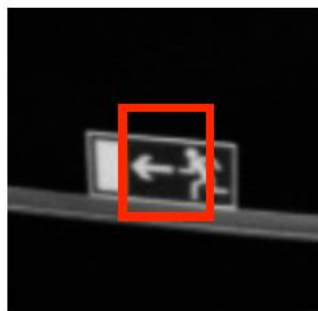
Ideal Properties of a Detector/Descriptor

Rotation invariance

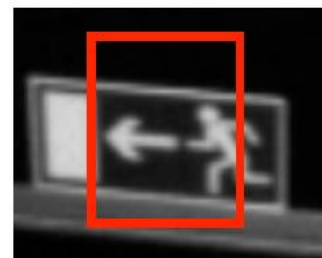
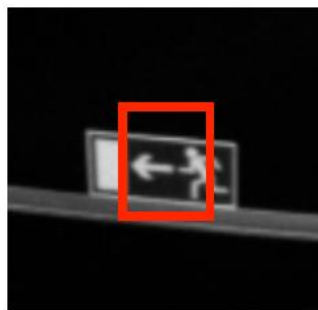
(more generally:
Viewpoint invariance)



Illumination invariance

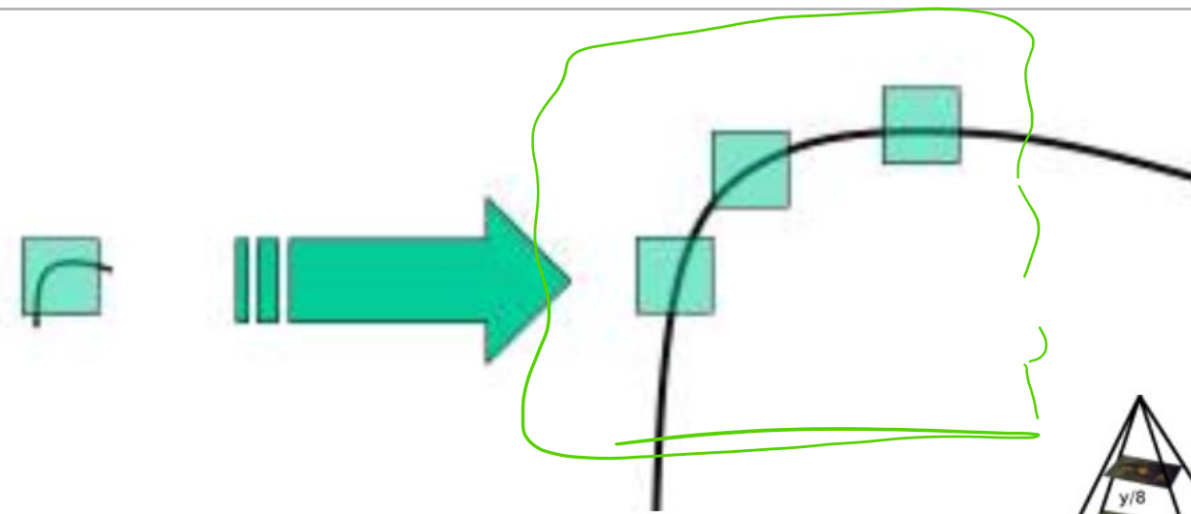


Scale invariance

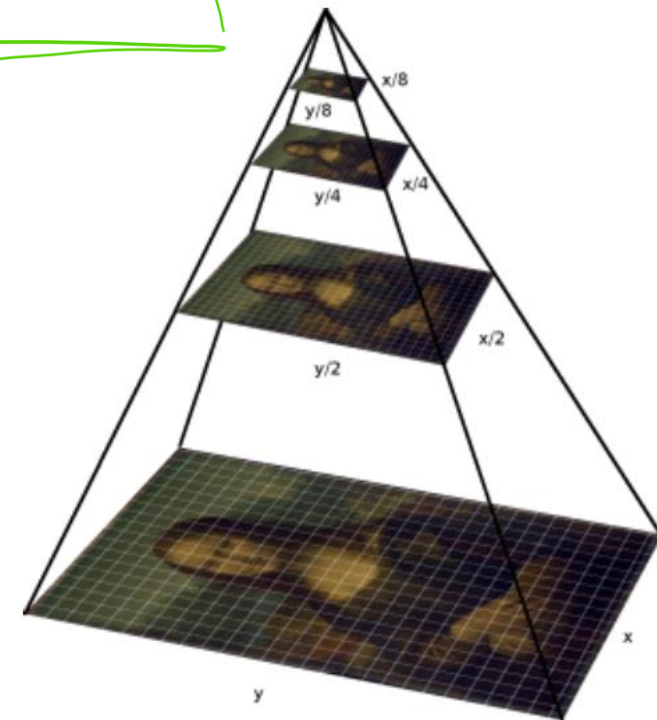


(more in the Lab 5 handouts: repeatability, efficiency ..)

Are Harris Corners **Scale** invariant?



- **Other detectors have been proposed:**
huge literature:
SIFT, SURF, ORB, BRIEF, MSER, ...
- **blob detectors:**
process the image at different scales



Nice tutorial on SIFT:

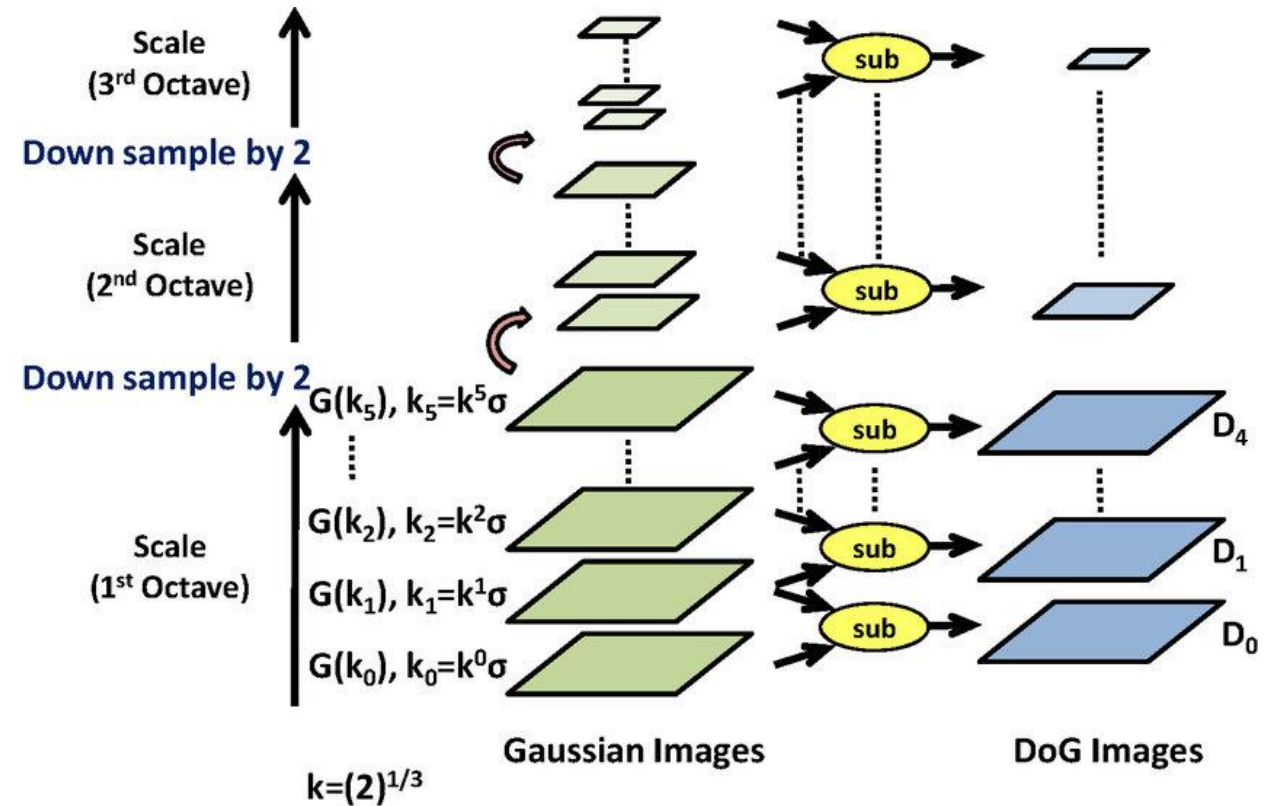
[\(242\) Lecture 05 - Scale-invariant Feature Transform \(SIFT\) - YouTube](#)

SIFT – Scale Invariant Feature Transform

- Goal: Find a scale and orientation invariant feature
- SIFT Steps:
 - Find the interest points using LoG (Laplacian of Gradient) which is approximated by DoG (Difference of Gaussian) → called scale space peak selection
 - Reject the outliers and finalize keypoints/interest points using second-order Hessian matrix
 - Assign Orientation to each keypoint
 - Create SIFT descriptor (vector of 128 dimension) for each key point

Gaussian Blurring

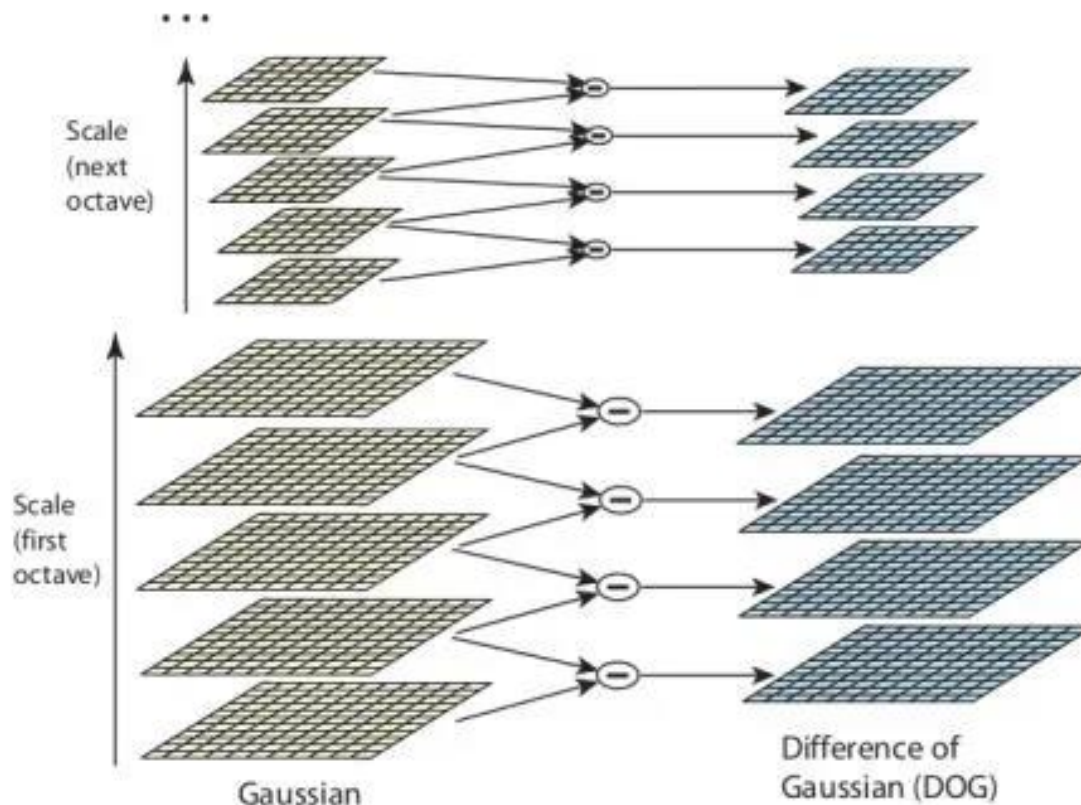
- In imaging science, **difference of Gaussians (DoG)** is a feature enhancement algorithm that involves the subtraction of one Gaussian blurred version of an original image from another, less blurred version of the original (source: Wikipedia)
- Within an octave, images are progressively blurred using the Gaussian Blur operator.



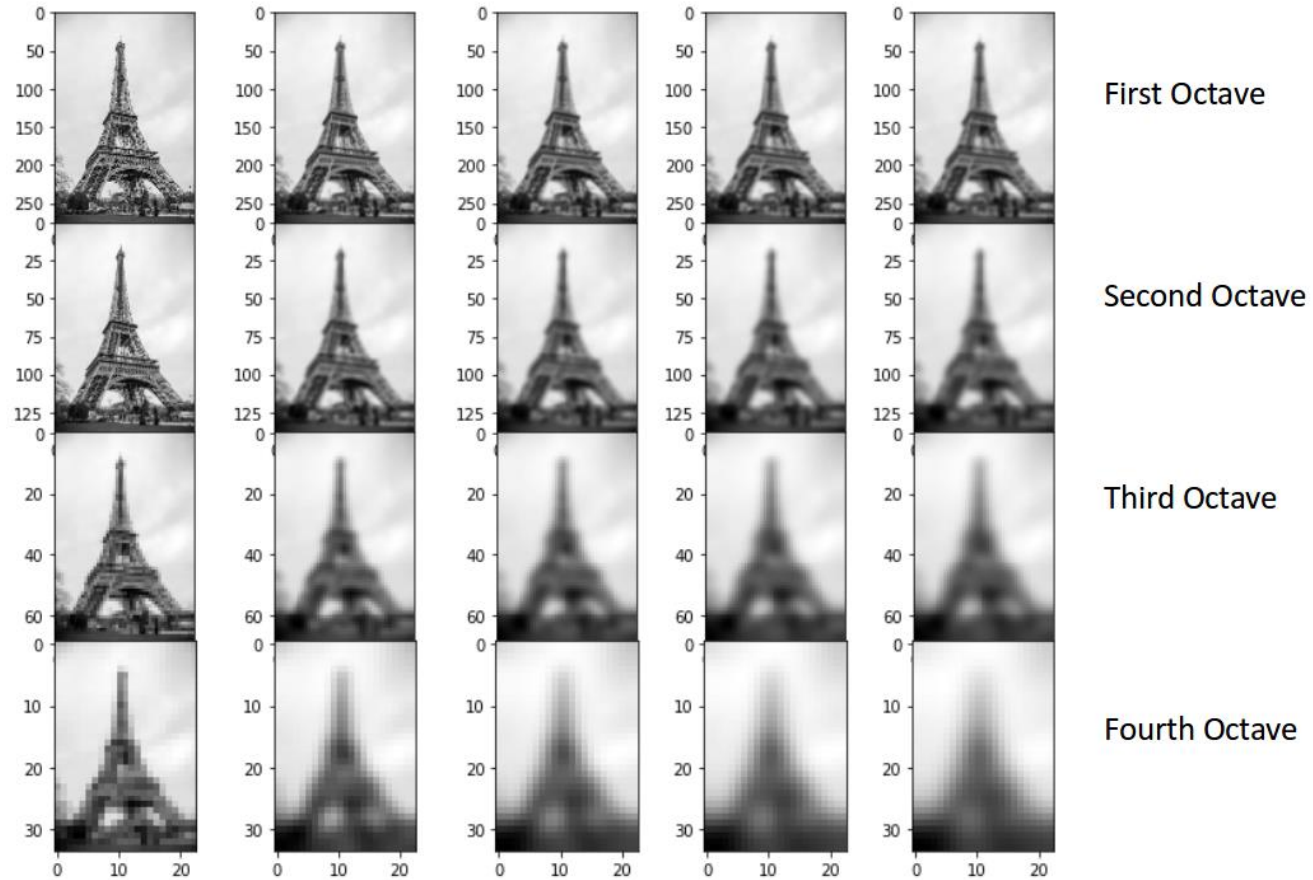
First step: find keypoints - continued

- **DOG(Difference of Gaussian kernel)**

The difference of Gaussian is obtained as the difference of Gaussian blurring of an image with two different σ



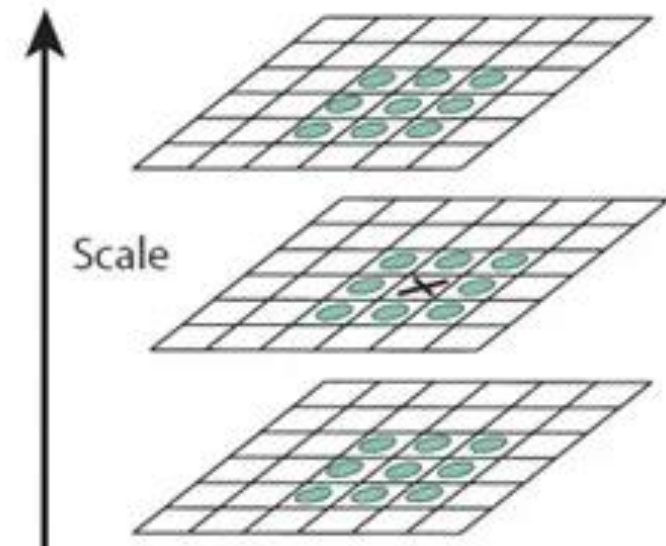
Example on DoG steps



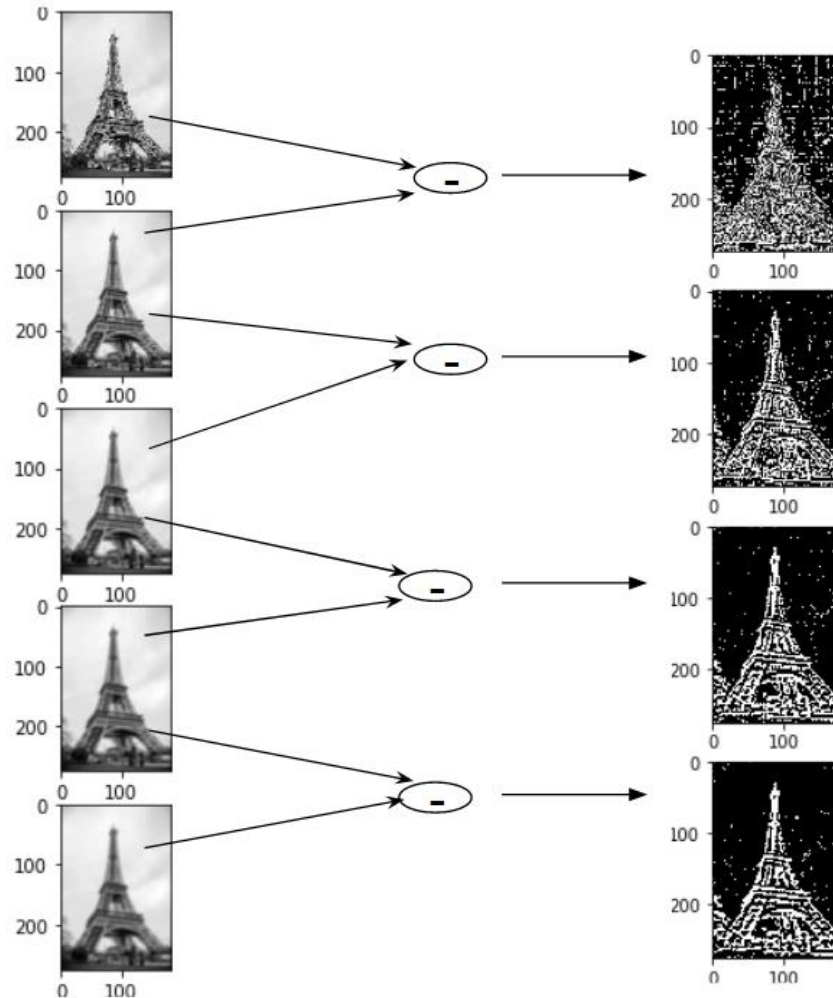
Finding keypoints

- One pixel in an image is compared with its 8 neighbors as well as 9 pixels in the next scale and 9 pixels in previous scales (total of 26 pixels)
- If it is a local extrema(has the maximum or minimum intensity), it is a potential keypoint
- Issue: some keypoints are outlier
- Next step: remove outliers using hessian

<https://www.analyticsvidhya.com/blog/2019/10/detailed-guide-powerful-sift-technique-image-matching-python/>



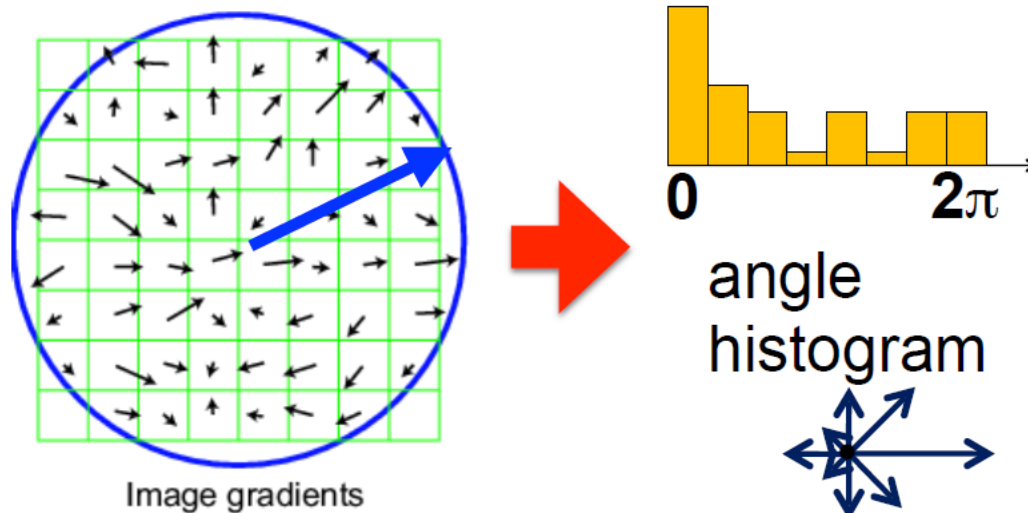
Example on DoG steps



Example: SIFT Descriptor (1/2)

SIFT: Scale-Invariant Feature Transform

- Take 16x16 square window around detected feature
- Compute gradient orientation and magnitude for each pixel
- Create histogram of gradients weighted by magnitude
- Peak is **orientation** of feature



© Dr. Mubarak Shah. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>



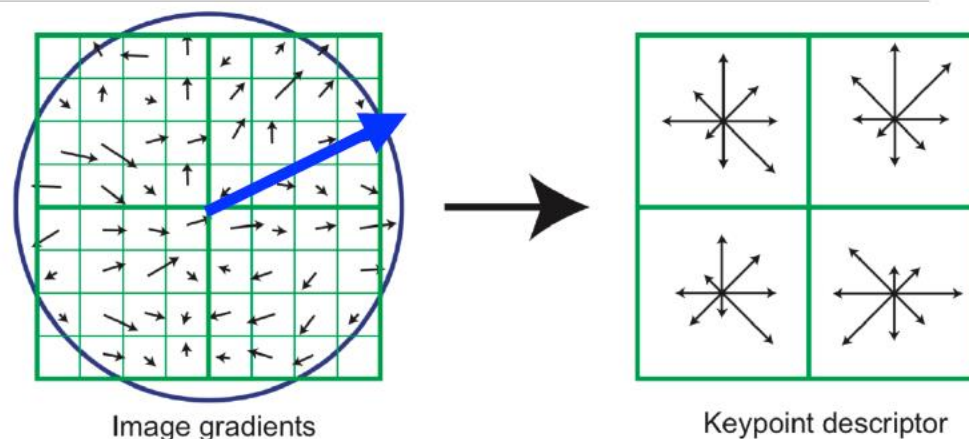
Lowe, David G. (1999). "Object recognition from local scale-invariant features", CVPR'99

D. G. Lowe, "Object recognition from local scale-invariant features," Proceedings of the Seventh IEEE International Conference on Computer Vision, Kerkira, Greece, 1999, pp. 1150-1157 vol.2, doi: 10.1109/ICCV.1999.790410 © IEEE. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>

Example: SIFT Descriptor (2/2)

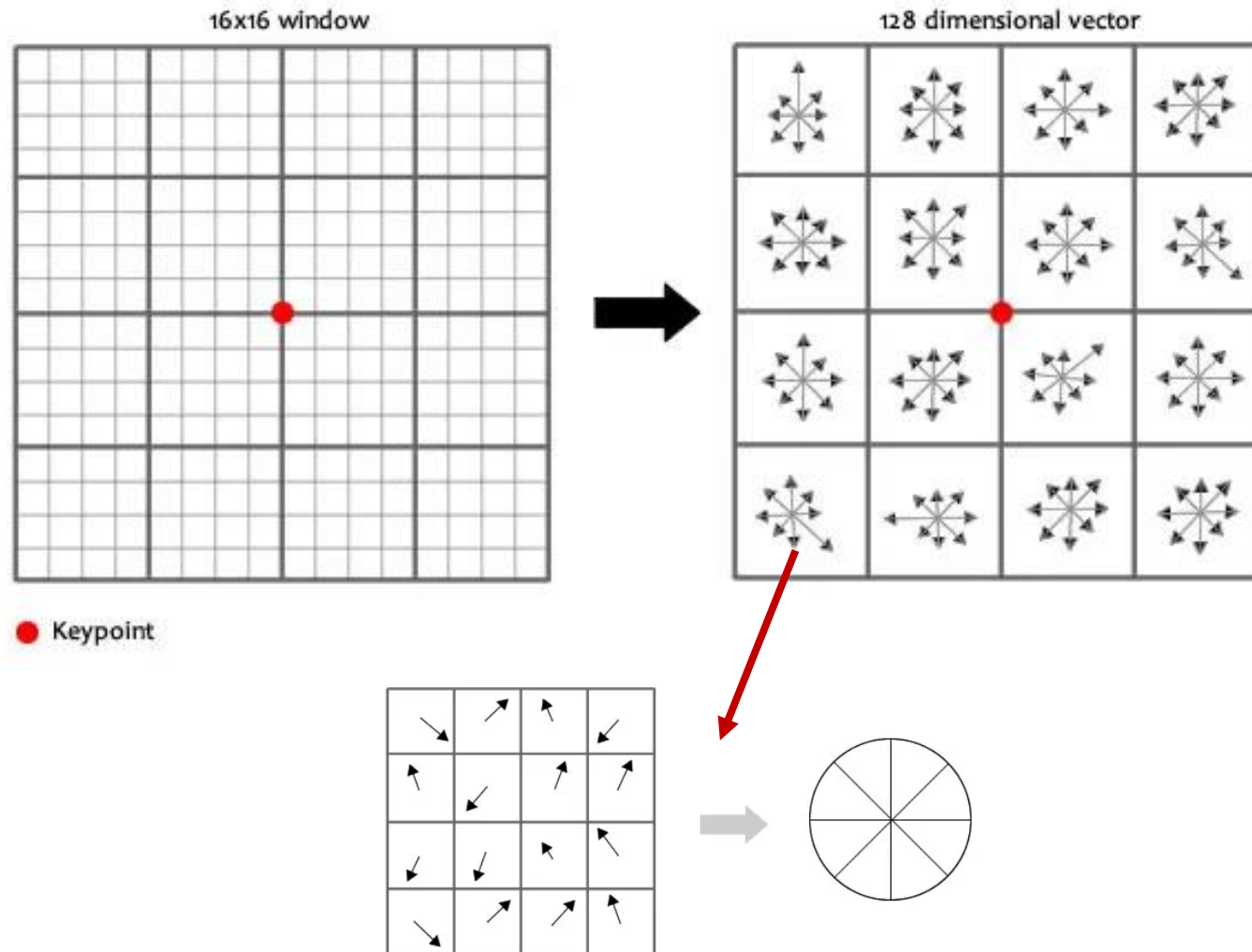
How to get SIFT descriptor?

- Transform all gradients with respect to (main) orientation
- Split window in 16 squares and for each compute a histogram with 8 sectors
- Stack histogram into a **descriptor** vector of $16 \times 8 = 128$ scalars
- Normalize to have norm = 1



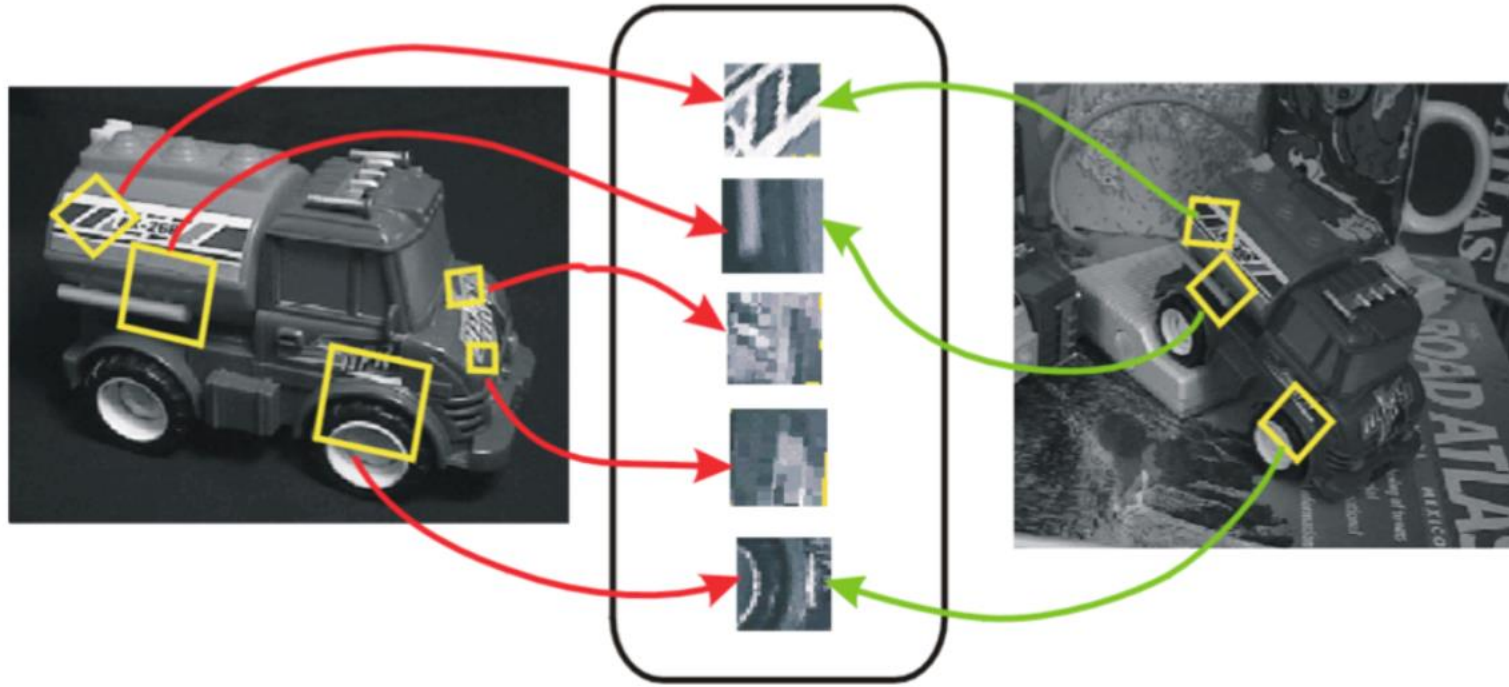
Lowe, David G. (1999). "Object recognition from local scale-invariant features", CVPR'99

Looking closer to SIFT dimension...



$$16 \times 8 = 128$$

Feature Matching



© Frank Dellaert. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>

- For each descriptor in I_1 find closest descriptor in I_2 (nearest neighbor)
- Speed up with **approximate** nearest neighbor algorithms (FLANN library)

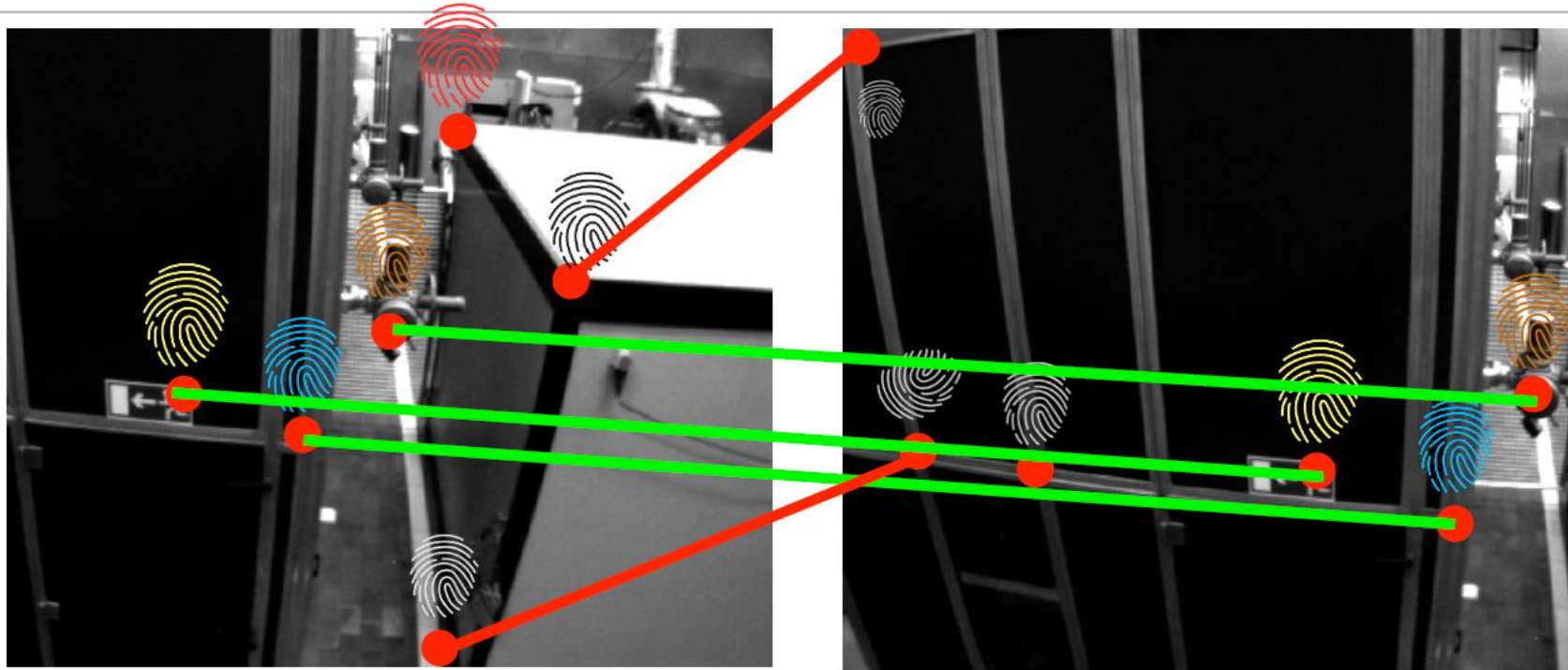
Some references on nearest neighbor algorithm

- [https://en.wikipedia.org/wiki/Nearest neighbour algorithm](https://en.wikipedia.org/wiki/Nearest_neighbour_algorithm)
- <https://scikit-learn.org/stable/modules/neighbors.html>



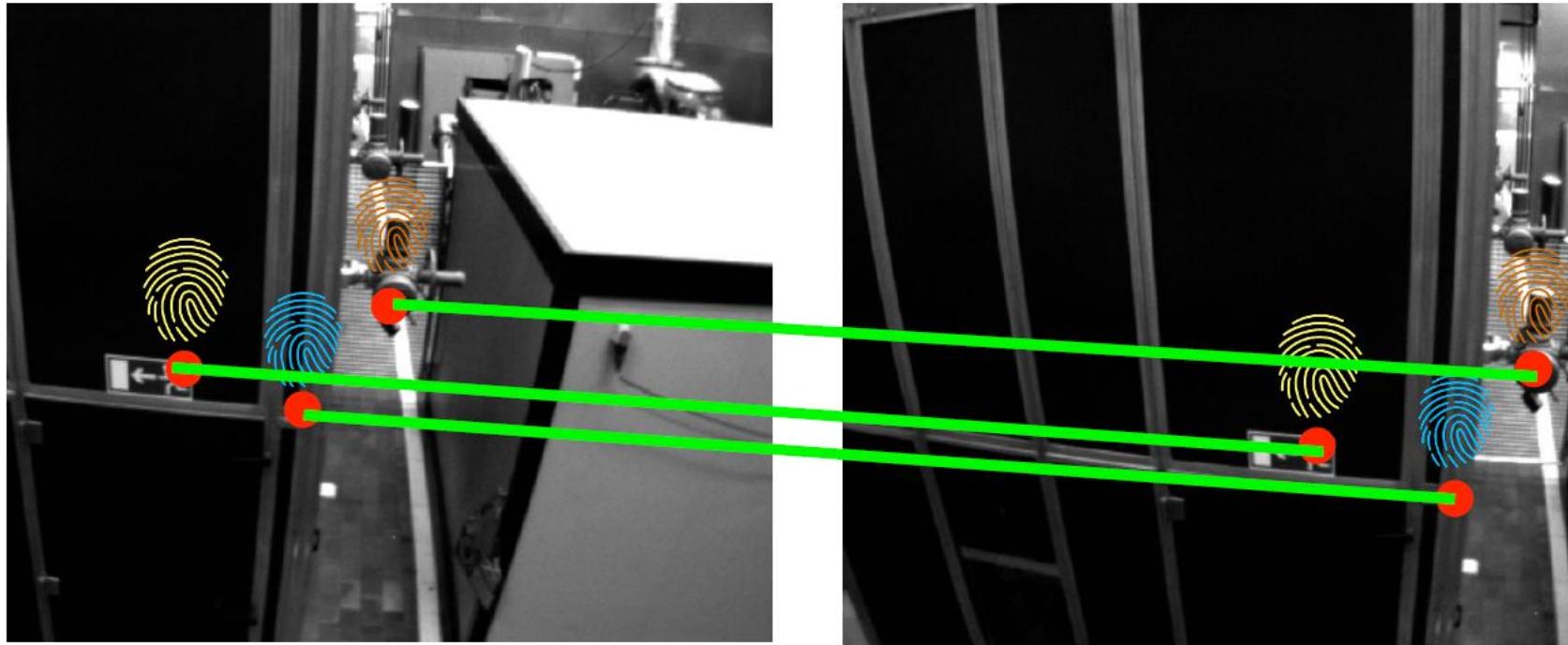
2-View Geometry

2-view Geometry



Question: can we estimate the motion of the camera between I_1 and I_2 using pixel correspondences?

2-view Geometry

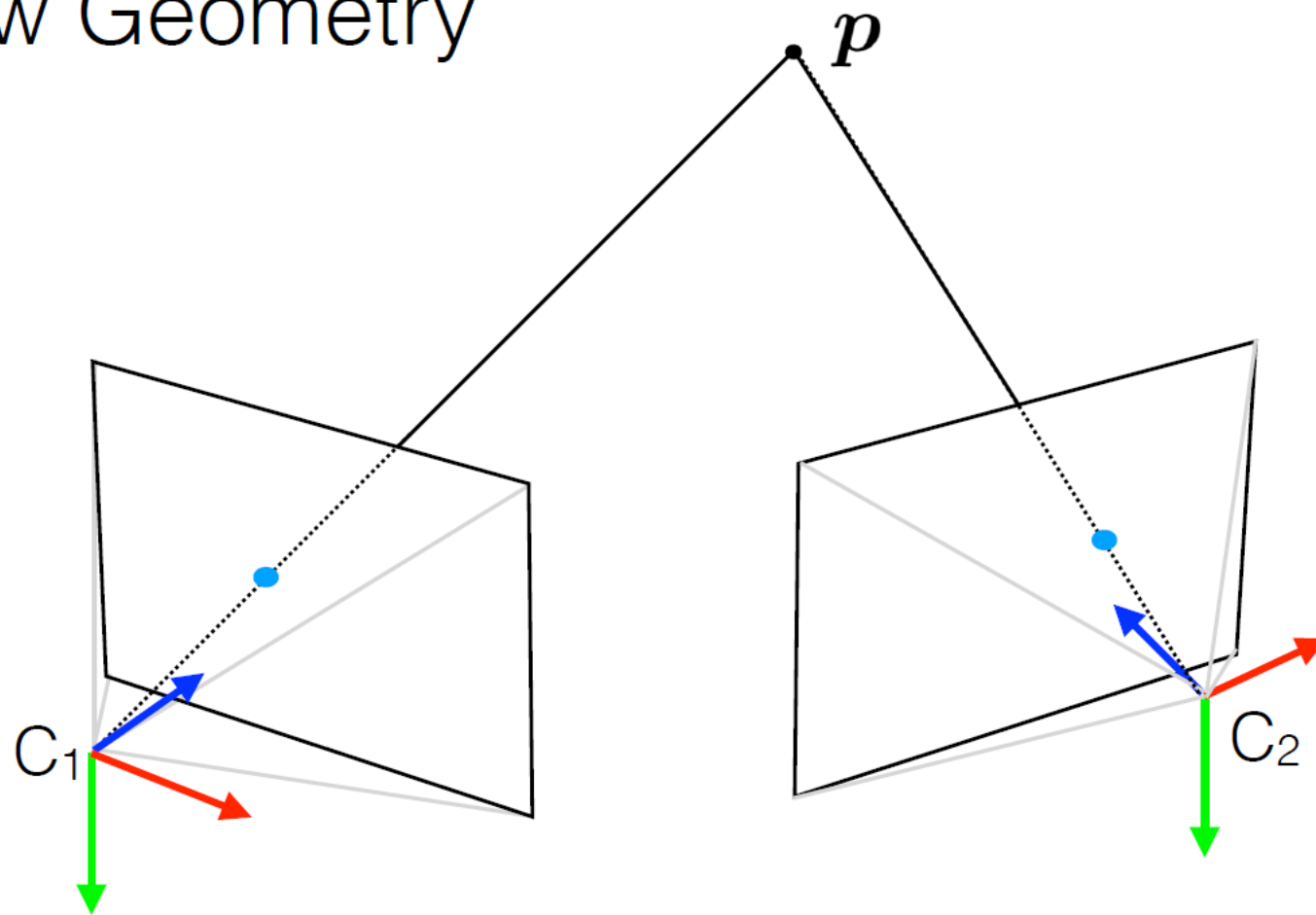


Question: can we estimate the motion of the camera between I_1 and I_2 using pixel correspondences?

Today's assumptions:

- no wrong correspondences (outliers)
- 3D point is not moving
- camera calibration is known

2-view Geometry



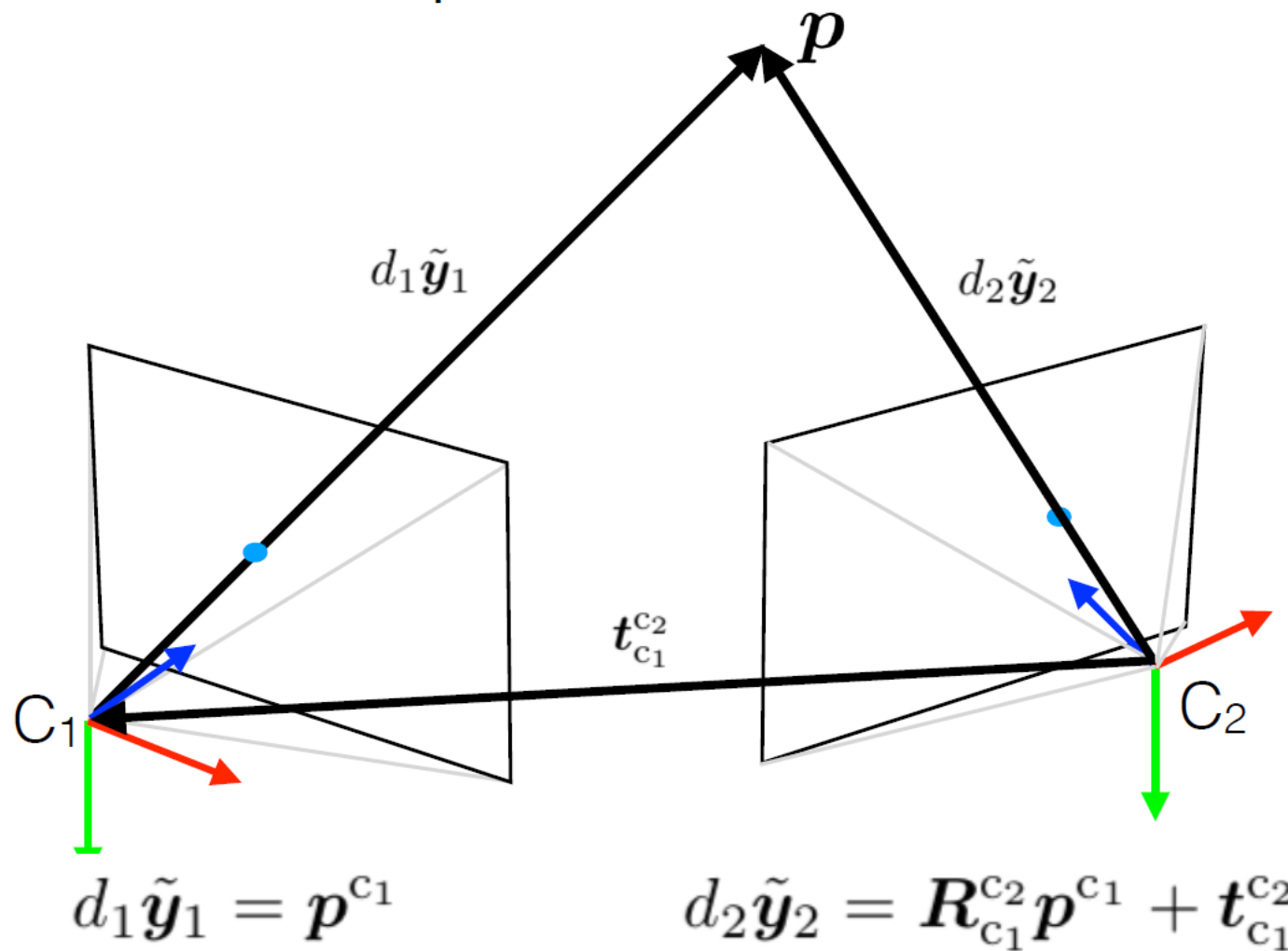
$$p_z^{C_1} \tilde{\mathbf{x}}_1 = \mathbf{K}_1 [\mathbf{R}_W^{C_1} \mathbf{t}_W^{C_1}] \tilde{\mathbf{p}}^W$$

$$\mathbf{K}_1 = \begin{bmatrix} s_{x_1} f_1 & s_{\theta_1} f_1 & o_{x_1} \\ 0 & s_{y_1} f_1 & o_{y_1} \\ 0 & 0 & 1 \end{bmatrix}$$

$$p_z^{C_2} \tilde{\mathbf{x}}_2 = \mathbf{K}_2 [\mathbf{R}_W^{C_2} \mathbf{t}_W^{C_2}] \tilde{\mathbf{p}}^W$$

$$\mathbf{K}_2 = \begin{bmatrix} s_{x_2} f_2 & s_{\theta_2} f_2 & o_{x_2} \\ 0 & s_{y_2} f_2 & o_{y_2} \\ 0 & 0 & 1 \end{bmatrix}$$

Geometric Interpretation



$$(t_{C_1}^{C_2} \times \tilde{y}_2) \perp (R_{C_1}^{C_2} \tilde{y}_1) \iff (t_{C_1}^{C_2} \times \tilde{y}_2)^T R_{C_1}^{C_2} \tilde{y}_1 = 0$$

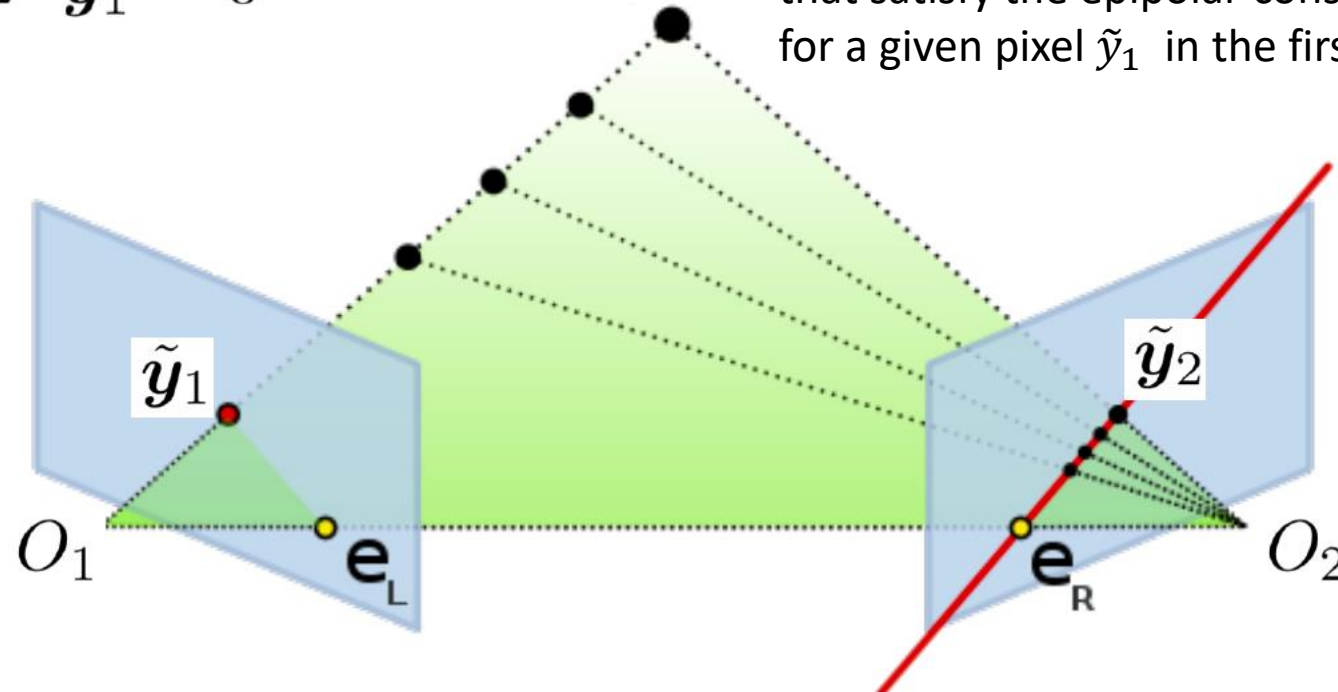
$$\iff ([t_{C_1}^{C_2}]_{\times} \tilde{y}_2)^T R_{C_1}^{C_2} \tilde{y}_1 = 0 \iff \tilde{y}_2^T [t_{C_1}^{C_2}]_{\times} R_{C_1}^{C_2} \tilde{y}_1 = 0$$

Epipolar Geometry

Epipolar Constraint:

$$\tilde{\mathbf{y}}_2^T \mathbf{E} \tilde{\mathbf{y}}_1 = 0$$

Epipolar line: Line corresponding to a set of pixels $\tilde{\mathbf{y}}_2$ in the second image that satisfy the epipolar constraints for a given pixel $\tilde{\mathbf{y}}_1$ in the first image



Plane passing through the optical centers and point p



epipolar plane



epipolar line

e_L , e_R : epipoles

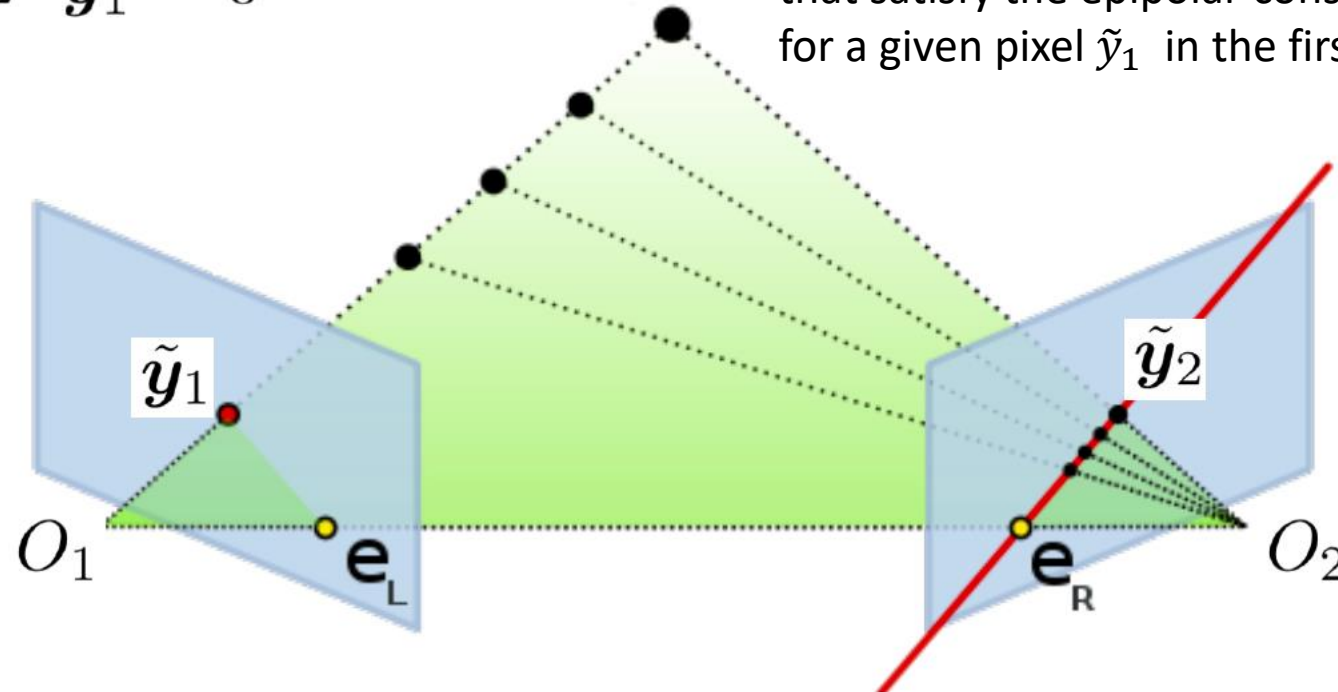
Plane passing through the optical centers and point p

Epipolar Geometry

Epipolar Constraint:

$$\tilde{\mathbf{y}}_2^T \mathbf{E} \tilde{\mathbf{y}}_1 = 0$$

Epipolar line: Line corresponding to a set of pixels $\tilde{\mathbf{y}}_2$ in the second image that satisfy the epipolar constraints for a given pixel $\tilde{\mathbf{y}}_1$ in the first image



Plane passing through the optical centers and point p

 epipolar plane  epipolar line $\mathbf{e}_L, \mathbf{e}_R$: epipoles

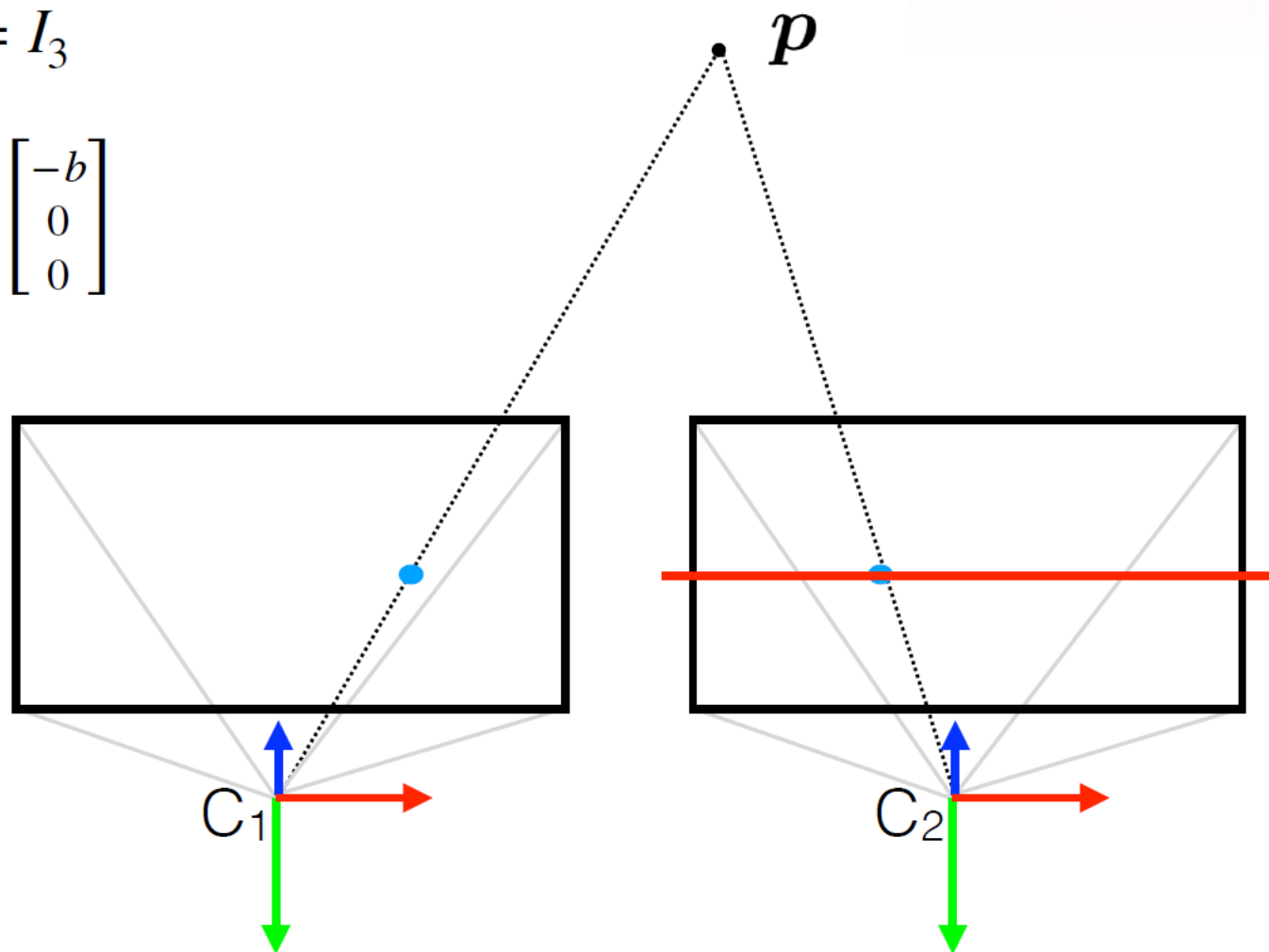
Plane passing through the optical centers and point p

Example: Stereo Camera



$$R_{C_1}^{C_2} = I_3$$

$$t_{C_1}^{C_2} = \begin{bmatrix} -b \\ 0 \\ 0 \end{bmatrix}$$



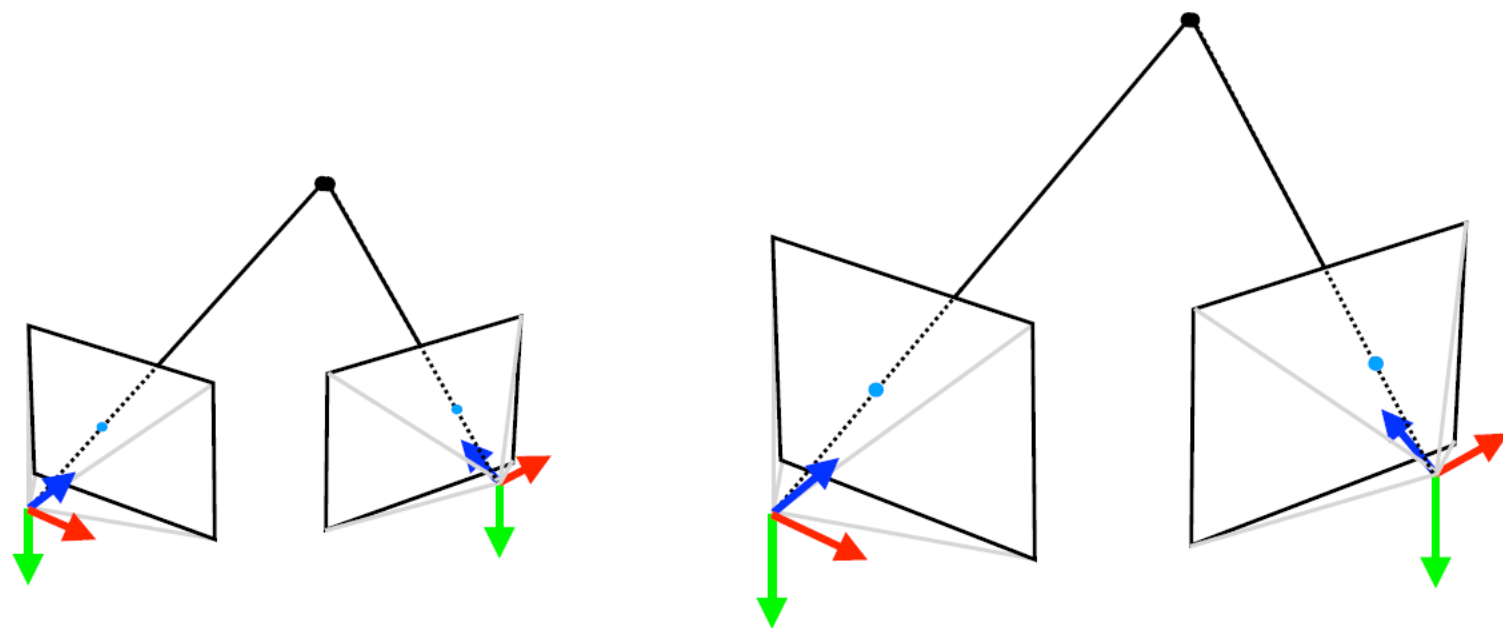
also: easy to triangulate points given geometry

Estimating Poses from Correspondences

Given N calibrated pixel correspondences:

$$(\tilde{\mathbf{y}}_{1,k}, \tilde{\mathbf{y}}_{2,k}) \text{ for } k = 1, \dots, N$$

compute the relative pose between the cameras



Can we estimate the scale of the translation (baseline)?

Estimating Poses from Correspondences

Given N calibrated pixel correspondences:

$$(\tilde{\mathbf{y}}_{1,k}, \tilde{\mathbf{y}}_{2,k}) \text{ for } k = 1, \dots, N$$

1. leverage the epipolar constraints to estimate the essential matrix $\mathbf{E}$

$$\tilde{\mathbf{y}}_{2,k}^T \mathbf{E} \tilde{\mathbf{y}}_{1,k} = 0$$

2. Retrieve the rotation and translation (up to scale) from the $\mathbf{E}$

$$\mathbf{E} = [\mathbf{t}]_{\times} \mathbf{R}$$

Estimating Poses from Correspondences

Given N calibrated pixel correspondences:

$$(\tilde{\mathbf{y}}_{1,k}, \tilde{\mathbf{y}}_{2,k}) \text{ for } k = 1, \dots, N$$

1. leverage the epipolar constraints to estimate the essential matrix $\mathbf{E}$

$$\tilde{\mathbf{y}}_{2,k}^T \mathbf{E} \tilde{\mathbf{y}}_{1,k} = 0$$

2. Retrieve the rotation and translation (up to scale) from the $\mathbf{E}$

$$\mathbf{E} = [\mathbf{t}]_{\times} \mathbf{R}$$

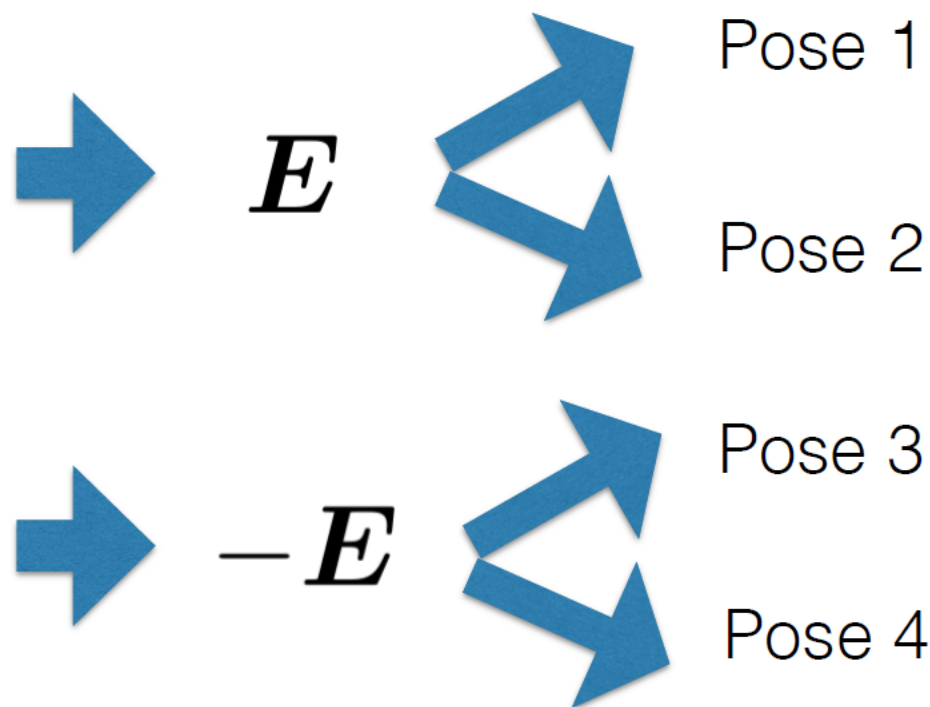
Retrieving Pose from Essential Matrix

Theorem 1 (Pose recovery from essential matrix, Thm 5.7 in [1]). *There exist exactly two relative poses $(\mathbf{R}, \mathbf{t})$ with $\mathbf{R} \in \text{SO}(3)$ and $\mathbf{t} \in \mathbb{R}^3$ corresponding to a nonzero essential matrix $\mathbf{E}$ (i.e., such that $\mathbf{E} = [\mathbf{t}]_{\times} \mathbf{R}$):*

$$\mathbf{t}_1 = \mathbf{U} \mathbf{R}_z(+\pi/2) \mathbf{\Sigma} \mathbf{U}^{\top} \quad \mathbf{R}_1 = \mathbf{U} \mathbf{R}_z(+\pi/2) \mathbf{V}^{\top} \quad (13.19)$$

$$\mathbf{t}_2 = \mathbf{U} \mathbf{R}_z(-\pi/2) \mathbf{\Sigma} \mathbf{U}^{\top} \quad \mathbf{R}_2 = \mathbf{U} \mathbf{R}_z(-\pi/2) \mathbf{V}^{\top} \quad (13.20)$$

where $\mathbf{E} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^{\top}$ is the singular value decomposition of the matrix $\mathbf{E}$, and $\mathbf{R}_z(+\pi/2)$ is an elementary rotation around the z -axis of an angle $\pi/2$.



Steps for Stereo Calibration Process

-) Epipolar Geometry
-) Estimating Fundamental Matrix
-) Finding Dense Correspondences
-) Computing Depth

Stereo Calibration Procedure

Find a set of **corresponding features** in left and right images (e.g. using SIFT or hand-picked)

Left image



Right image



Stereo Calibration Procedure

Find a set of **corresponding features** in left and right images (e.g. using SIFT or hand-picked)

Left image



$$\text{green circle } (\mathbf{u}_l^{(1)}, \mathbf{v}_l^{(1)})$$

$\vdots$

$$\text{yellow circle } (\mathbf{u}_l^{(m)}, \mathbf{v}_l^{(m)})$$

Right image



$$\text{green circle } (\mathbf{u}_r^{(1)}, \mathbf{v}_r^{(1)})$$

$\vdots$

$$\text{yellow circle } (\mathbf{u}_r^{(m)}, \mathbf{v}_r^{(m)})$$

Stereo Calibration Procedure

Step A: For each correspondence i , write out epipolar constraint.

$$\underbrace{\begin{bmatrix} u_l^{(i)} & v_l^{(i)} & 1 \end{bmatrix}}_{\text{Known}} \underbrace{\begin{bmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{bmatrix}}_{\text{Unknown}} \underbrace{\begin{bmatrix} u_r^{(i)} \\ v_r^{(i)} \\ 1 \end{bmatrix}}_{\text{Known}} = 0$$

Stereo Calibration Procedure

Step A: For each correspondence i , write out epipolar constraint.

$$\underbrace{\begin{bmatrix} u_l^{(i)} & v_l^{(i)} & 1 \end{bmatrix}}_{\text{Known}} \underbrace{\begin{bmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{bmatrix}}_{\text{Unknown}} \underbrace{\begin{bmatrix} u_r^{(i)} \\ v_r^{(i)} \\ 1 \end{bmatrix}}_{\text{Known}} = 0$$

Expand the matrix to get linear equation:

$$(f_{11}u_r^{(i)} + f_{12}v_r^{(i)} + f_{13})u_l^{(i)} + (f_{21}u_r^{(i)} + f_{22}v_r^{(i)} + f_{23})v_l^{(i)} + f_{31}u_r^{(i)} + f_{32}v_r^{(i)} + f_{33} = 0$$

Step B: Rearrange terms to form a linear system.

$$\begin{bmatrix}
 u_l^{(1)} & u_r^{(1)} & u_l^{(1)} v_r^{(1)} & u_l^{(1)} & v_l^{(1)} u_r^{(1)} & v_l^{(1)} v_r^{(1)} & v_l^{(1)} & u_r^{(1)} & v_r^{(1)} & 1 \\
 \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\
 u_l^{(i)} & u_r^{(i)} & u_l^{(i)} v_r^{(i)} & u_l^{(i)} & v_l^{(i)} u_r^{(i)} & v_l^{(i)} v_r^{(i)} & v_l^{(i)} & u_l^{(i)} & u_r^{(i)} & 1 \\
 \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\
 u_l^{(m)} & u_r^{(m)} & u_l^{(m)} v_r^{(m)} & u_l^{(m)} & v_l^{(m)} u_r^{(m)} & v_l^{(m)} v_r^{(m)} & v_l^{(m)} & u_l^{(m)} & u_r^{(m)} & 1
 \end{bmatrix}
 \begin{bmatrix}
 f_{11} \\
 f_{21} \\
 f_{31} \\
 f_{22} \\
 f_{23} \\
 f_{31} \\
 f_{32} \\
 f_{33}
 \end{bmatrix}
 =
 \begin{bmatrix}
 0 \\
 \vdots \\
 0 \\
 \vdots \\
 0
 \end{bmatrix}$$

A
 (Known)

$\mathbf{f}$
 (Unknown)

Fundamental matrix acts on homogenous coordinates.

$$[u_l \quad v_l \quad 1] \begin{bmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{bmatrix} \begin{bmatrix} u_r \\ v_r \\ 1 \end{bmatrix} = 0 = [u_l \quad v_l \quad 1] \begin{bmatrix} kf_{11} & kf_{12} & kf_{13} \\ kf_{21} & kf_{22} & kf_{23} \\ kf_{31} & kf_{32} & kf_{33} \end{bmatrix} \begin{bmatrix} u_r \\ v_r \\ 1 \end{bmatrix}$$

Fundamental Matrix F and kF describe the same epipolar geometry. That is, F is defined only up to a scale.

Set Fundamental Matrix to some arbitrary scale.

$$\|\mathbf{f}\|^2 = 1$$

Step C: Find least squares solution for fundamental matrix F

We want Af as close to 0 as possible and $\|f\|^2 = 1$:

$$\min_f \|Af\|^2 \quad \text{such that } \|f\|^2 = 1$$

Constrained linear least squares problem

Like solving Projection Matrix during Camera Calibration.

Or, Homography Matrix for Image Stitching.

Rearrange solution f to form the fundamental matrix F .

Extracting Rotation and Translation

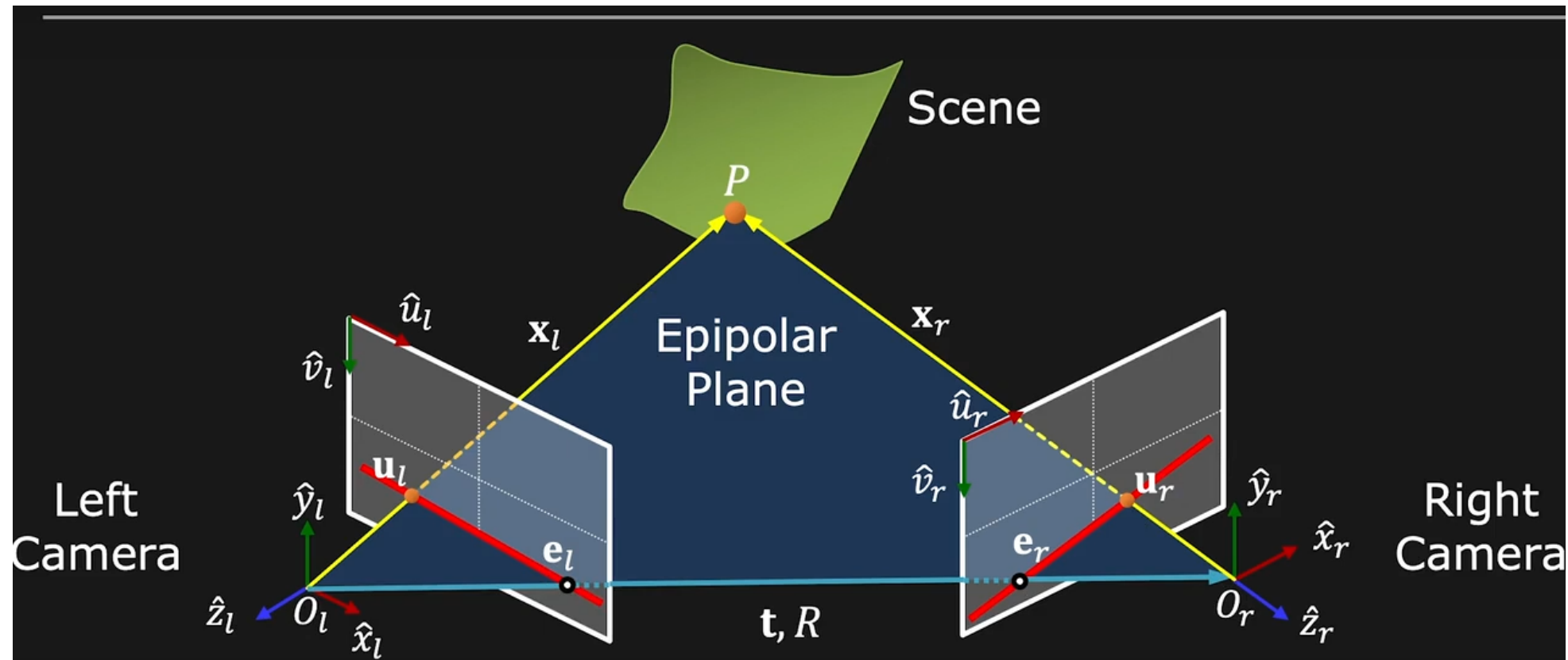
Step D: Compute essential matrix E from known left and right intrinsic camera matrices and fundamental matrix F .

$$E = K_l^T F K_r$$

Step E: Extract R and t from E .

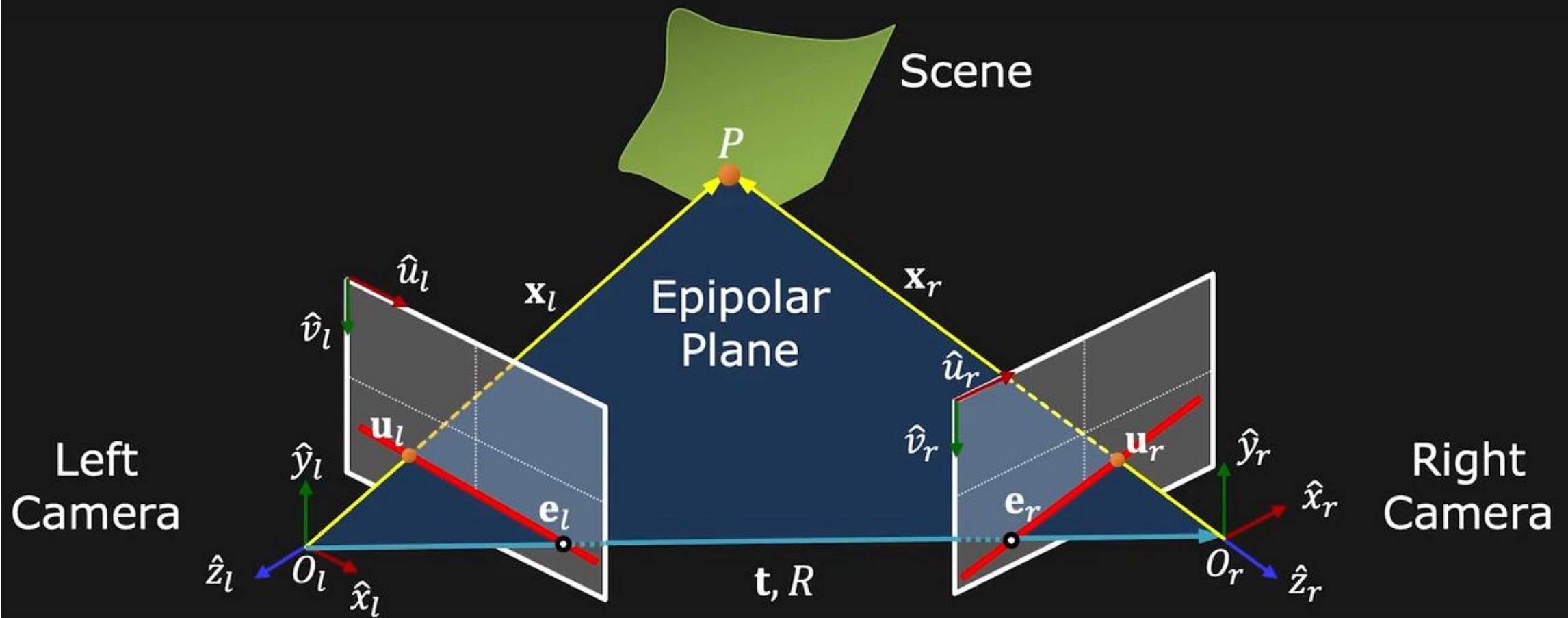
$$E = T_{\times} R$$

(Using Singular Value Decomposition)



Epipolar Line: Intersection of image plane and epipolar plane.

Every scene point has **two corresponding epipolar lines**, one each on the two image planes.



Given a point in one image, the corresponding point in the other image must lie on the epipolar line.

Given: Fundamental matrix F and point on left image (u_l, v_l)

Find: Equation of Epipolar line in the right image

Epipolar Constraint Equation:

$$[u_l \quad v_l \quad 1] \begin{bmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{bmatrix} \begin{bmatrix} u_r \\ v_r \\ 1 \end{bmatrix} = 0$$

Expanding the matrix equation gives:

$$(f_{11}u_l + f_{21}v_l + f_{31})u_r + (f_{12}u_l + f_{22}v_l + f_{32})v_r + (f_{13}u_l + f_{23}v_l + f_{33}) = 0$$

Equation for **right epipolar line**: $a_l u_r + b_l v_r + c_l = 0$

Find Epipolar Lines

Given the Fundamental matrix,

$$F = \begin{bmatrix} -.003 & -.028 & 13.19 \\ -.003 & -.008 & -29.2 \\ 2.97 & 56.38 & -9999 \end{bmatrix}$$

Left Image



Right Image



Find Epipolar Lines

Given the Fundamental matrix,

$$F = \begin{bmatrix} -.003 & -.028 & 13.19 \\ -.003 & -.008 & -29.2 \\ 2.97 & 56.38 & -9999 \end{bmatrix}$$

and the **left** image point

$$\tilde{\mathbf{u}}_l = \begin{bmatrix} 343 \\ 221 \\ 1 \end{bmatrix}$$

Left Image



Right Image



Find Epipolar Lines

Given the Fundamental matrix,

$$F = \begin{bmatrix} -.003 & -.028 & 13.19 \\ -.003 & -.008 & -29.2 \\ 2.97 & 56.38 & -9999 \end{bmatrix}$$

and the **left** image point

$$\tilde{\mathbf{u}}_l = \begin{bmatrix} 343 \\ 221 \\ 1 \end{bmatrix}$$

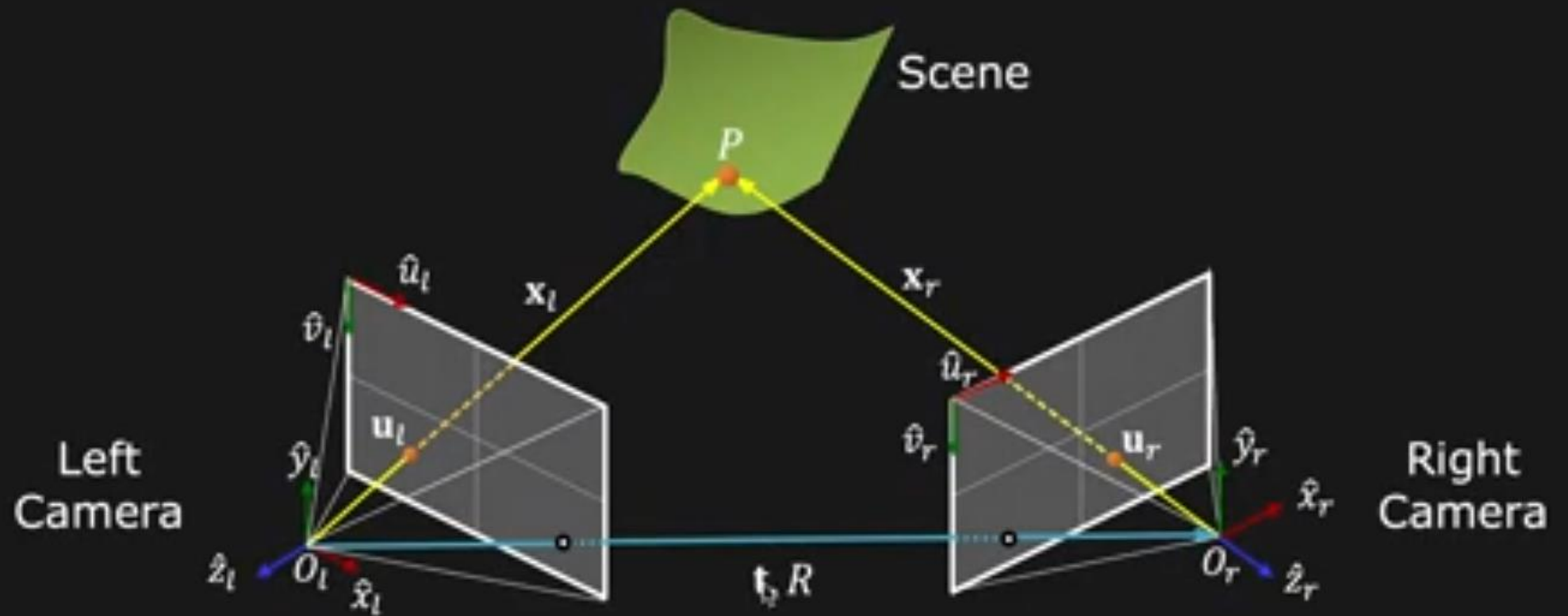
The equation for the epipolar line in the **right** image is

$$.03u_r + .99v_r - 265 = 0$$



Note

- If the camera calibration is unknown, we can compute Fundamental matrix.
- If the camera calibration is known, we compute the essential matrix



- ✓ 1. Assume Camera Matrix K is known for each camera
- ✓ 2. Find a few Reliable Corresponding Points
- ✓ 3. Find Relative Camera Position t and Orientation R
- ✓ 4. Find Dense Correspondence
- 5. Compute Depth using Triangulation